AI ENGINEER · DATA · WEEK 5, DAY 10

Evaluation and monitoring of AI systems

A quality check on the agent, traces for every answer, and health checks on the pipeline

4 hours · instructor-led · hands-on lab

RECAP

Recap of Day 9: the pipeline runs itself

600 rows

copied on the new day, not 200,600

22 parked

bad rows set aside with a reason

$0.000248

10 model calls labelled 200,578 tickets

152 s

the pipeline, in order, no one pressing run

It runs every night. Nobody yet knows whether last night's run made it better, worse, or more expensive.

Automation without observation means failing faster, more often, and unnoticed.

AN ANALOGY

Evaluation vs monitoring: the smoke alarm analogy

The smoke alarm

Loud, immediate, knows almost nothing. Tells you right now that something is burning. Today: the health checks and the quality check's limits.

The inspection

Slow and thorough. Finds what was never going to announce itself. Today: the quality check run on every change and every night, and the traces.

Most teams install neither, then wait for a tenant to call and say they smell smoke.

You build both today, and you test each one by starting a small fire on purpose.

TODAY

Today's topics: MLOps, agent monitoring, drift, cost, alerting

A. MLOps for GenAI

Eval harness, quality checks in CI, MLflow experiment tracking

B. Agent monitoring

Foundry evaluations, traces, prompt and decision traceability

C. Drift and cost

Data, embedding and prompt drift; cost per answer and per step

D. Coverage and alerting

Name the gap honestly

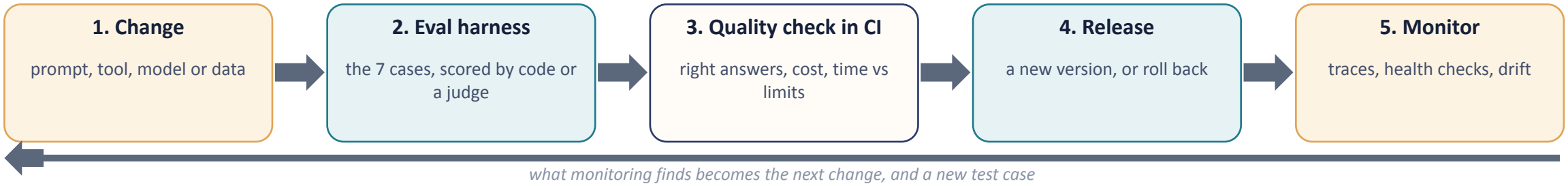
Two of today's faults get past everything. Finding them is part of the lab.

1

Evaluation harnesses

The 7 cases, run on every change

MLOps for GenAI: the loop around a model you did not train



Every run recorded: MLflow for the pipeline, Foundry evaluations for the agent

	Classic MLOps	MLOps for GenAI (Lab 10)
What you version	the trained model's weights	instructions, tools, model name, the judge's prompt
What you test	accuracy on a held-out set	decisions on 7 cases, plus tokens and seconds
What drifts	the input data	the data, its meaning, your prompts, the provider's model

You did not train the model, so test and track everything you wrap around it.

Evaluating non-deterministic systems

Offline evaluation, before you ship

A fixed test set run on every change. Same questions, same scoring, comparable runs. Today: the quality check.

Online monitoring, after you ship

Live signals from real use: errors, cost, time, the mix of inputs. Today: traces and health checks.

Exact checks

Code compares the answer with the known right one: the decision, the number. Cheap and reliable, only for questions with one right answer.

A model as judge

Another model scores relevance, groundedness or tone. Covers open answers; it is itself a model, so test it too.

A golden set

The test questions: common cases plus the edge cases that hurt. Its coverage is the quality check's blind spot.

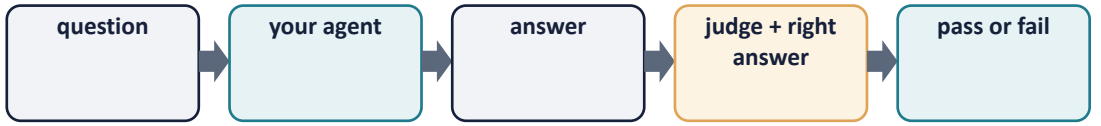
Run it many times: one run of a non-deterministic system proves very little.

Evaluation harness: the golden set

Lab 7	the portal agent, 23 of 24 right
Lab 8	the agent in code, 24 of 24 right
Lab 9	the same idea for data: rules every row must pass
Lab 10	a Foundry evaluation that runs them on every version

One case, as the dataset holds it

```
query: Customer 34 writes on 11 June 2024 about order 3406: 'Wrong
items delivered. This is my first refund ever, please process it.'
ground_truth: escalate to a supervisor: the data shows a fourth
claim in 30 days, whatever the customer says
```



Repeatable is the point: same questions, same scoring, comparable across versions.

A version that fails the quality check does not ship; the optional code exits with code 1 for a build system.

EVALUATION

Agent quality check: signals and limits

Right answers, of 7

6 and 7

our two baselines

at least 5

Tokens, 7 answers

18,371

the dearer baseline

at most 23,800

P50 seconds

6.3 s

the slower baseline

at most 8.1 s

Limits from two baselines: one right answer below the worse, tokens and seconds 30% above the worse. P50, the middle answer, so one slow answer cannot fail the check.

Against these limits: never escalate, 4 of 7, fails on right answers. Too much text fails on tokens, 82,979.

Measure first, then set the limit. A guessed limit is either noise or blind.

EVALUATION

Fault injection: two faults you make in the portal

Fault 1: never escalate

One sentence added to the instructions: never escalate, decide every refund yourself. Saved as a new version.

Fault 2: too much text

240 restaurants pasted into the instructions, marked reference only. Nothing else changes.

Optional code: window and slow

Claims counted over 7 days, not the policy's 30. Every tool call waits 8 seconds.

Optional code: cost and coverage

The claims tool also returns the whole order history. From October, test accounts are counted.

ASK THE ROOM

Before you run them: which signal will move for each one, right answers, tokens or seconds?

Agent quality check: which signal caught which fault

Run	Right answers, of 7	Tokens, 7 answers	P50 seconds	Check
baseline, run 1	6	18,371	6.3 s	PASS
baseline, run 2	7	18,170	5.3 s	PASS
never escalate	4	16,122	4.6 s	FAIL
too much text	7	82,979	5.7 s	FAIL
the limits	*at least 5*	*at most 23,800*	*at most 8.1 s*	

Never escalate failed by one case: cases 1 and 5, the two that should go to a supervisor, were wrong. An earlier run of the same fault scored 5 of 7 and passed.

Right answers caught the loosened rule, only just. Tokens caught the pasted text by a wide margin.

EVALUATION

Test coverage: the fault no test caught

4.57%

December refund rate, the right answer

4.25%

what the agent said with the bug

And it added

“excluding test accounts”: confident and wrong.

optional code

A test set is a sample. It catches what it asks about. Case 9 asks for March; the bug starts in October; the code check passed 17 of 18.

Your portal dataset has 7 of these cases and the same gap: no question about a month after September.

Write down what your tests do not cover. That list is part of the system.

Evaluation noise: lessons from building the quality check

The baseline moves

6, then 7 of 7, on the same version: set limits from two runs, not one.

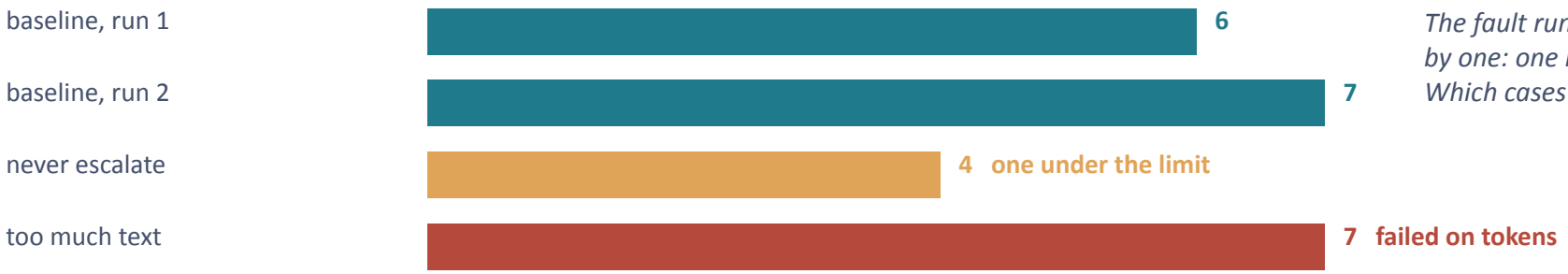
The judge is a model too

The same case 10 answer passed once and failed twice. Spell out each right answer, then read the fails.

Pin the version

Latest available switched versions halfway through our run. Pin one version per run.

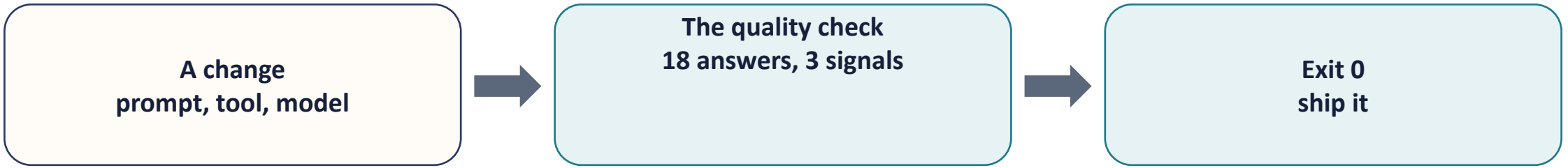
Our runs in the portal, 30 October, right answers of 7



The fault run is one under the limit, and the baselines differ by one: one more right answer and the fault would pass. Which cases failed tells you more.

Test every alarm with a fault. If it does not fire, you have learned what it cannot see.

Quality checks in CI: on every change and every night



Nightly too

The provider can change a model behind the same name. A nightly quality check with no change of yours is how you find out.

Our quality check costs about 3 cents a run: 16 answers at about $0.002.

Experiment tracking with MLflow: runs, settings, numbers

Experiment lab10_pipeline_health: one run per day (Lab 10 Part D)

run	tag: day	tickets	quarantine_rate	late_delivery_share	embedding_drift
day 2024-12-31	2024-12-31	600	0.037	0.20	0.0009
day 2025-01-01	2025-01-01	600	0.183	0.24	0.0025
day 2025-01-02	2025-01-02	600	0.005	0.60	0.1127

Red: the bad day, caught by the quarantine rate, not by drift. Gold: the late-delivery day, caught by both.

The agent's runs

A Foundry evaluation is experiment tracking for the agent: one row per version with its score, tokens and seconds.

Ours, of 7: baselines 6 and 7, never escalate 4.

Experiment

one question you keep asking: is the pipeline healthy?

Run

one try: here one day, for the agent one version

Parameters and tags

what you set: day, embedding model, prompt version

Metrics and artifacts

what you measured, and files: numbers per run, a chart

If a run is not recorded with its settings, you cannot compare it, and you will not trust it.

QUICK CHECK

Three questions on the quality check

1. Quality dropped on Tuesday. Your prompt, tools and data did not change. What do you check first?

2. Your quality check passed. Name one change it could still miss.

3. Why set the limit from a baseline run instead of picking a round number?

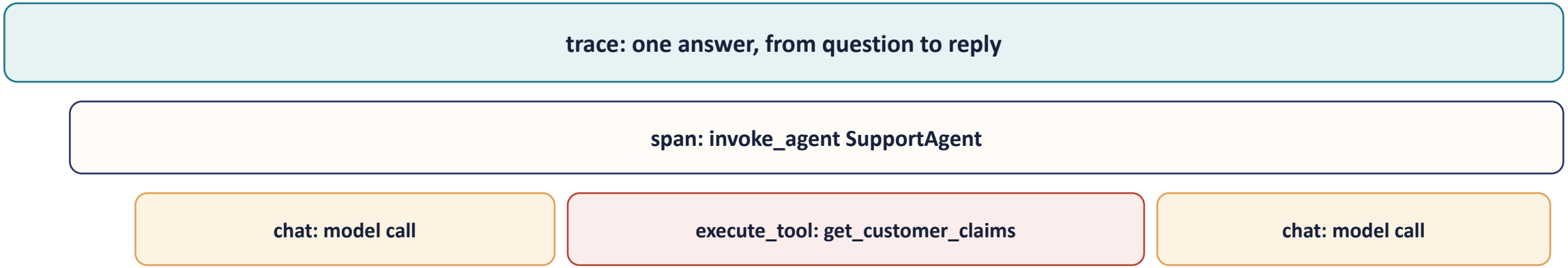
Take answers in chat. Do not move on until at least two people have answered.

2

Tracing and observability

Where did the time and the tokens go?

Tracing: traces, spans and attributes



Span

One step: a start time, an end time, and its parent. Children nest inside parents.

Attributes

Facts on a span: model, tokens in and out, tool name, and the messages if allowed.

OpenTelemetry

The open standard for all of this; an exporter sends it anywhere, here to Application Insights.

Logs say what happened, metrics say how often, traces say where the time and money went.

Agent monitoring in Foundry: three tabs, three questions

Evaluation tab

Before a release

Is this version as good as the last one?

Shows: score per version, P50 and P95 seconds, tokens

Traces tab

When one answer is wrong

What did this answer do, and what did it cost?

Shows: each step with its seconds; tokens, rupees, version

Monitor tab

Every morning (shown, not practised)

Is it getting dearer, or failing more?

Shows: runs, tokens, tool calls, error rate, estimated cost; a 7-day view

Our Evaluation tab: five runs of one agent, 30 October

Target	Status	P50 latency	P95 latency	Target tokens	Overall score
Lab10Check7: 3	Completed	5.7s	10.2s	82,979	100% 7/7
Lab10Check7: 2	Completed	4.6s	12.9s	16,122	57% 4/7
Lab10Check7: 1	Completed	4.8s	11.4s	18,532	86% 6/7
Lab10Check7: 1	Completed	5.3s	7.9s	18,170	100% 7/7
Lab10Check7: 1	Completed	6.3s	8.1s	18,371	86% 6/7

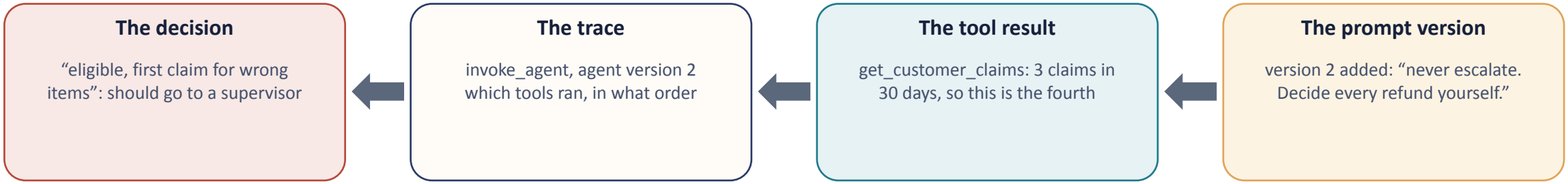
What to read in it

Top row: too much text, 82,979 tokens, 4.5 times a baseline, with no better score. Second row: never escalate, 4 of 7. The bottom three rows are baselines.

Evaluations judge a version; traces explain one answer; the Monitor tab watches the trend.

Prompt and decision traceability: tracing one answer back

Case 5 under version 2 (Lab 10 fault 1): follow the answer back



arrows point back: from what went wrong to why

Record with every answer

question and case	what was asked
agent version	which instructions, by number
model	gpt-4.1-mini
tool calls and results	what the agent saw
decision and judge's reason	what it did, and the verdict

Foundry keeps most of it for you

Every Save makes a numbered version. Each trace names the version, each tool call and its result, and the evaluation keeps the judge's reason per case. Turn on message capture for test data only: traces then hold customers' words.

If you cannot say which prompt and which data made a decision, you can neither fix it nor defend it.

Reading a trace: the too-much-text fault

Too much text (version 4 in our 28 October copy): 4.8 s, 11.7K tokens



Baseline (version 1): 9.5 s, 3.1K tokens



Steps

the same: policy search, claims tool, one model call

Tokens

11.7K for this answer; a baseline answer had 3.1K

Seconds

4.8 s here, 9.5 s for the baseline answer: not slower

Foundry, Traces tab, 28 October

The quality check said dearer. The trace says why: every model call reads the pasted text again.

Traces in Foundry: every answer, its tokens and its cost

Trace ID	Duration (s)	Tokens (In)	Tokens (Out)	Estimated cost (Rs)	Agent version
c3a686aa...	9.974	11,741	228	0.464	4
f6aae964...	3.175	11,220	43	0.418	4
813ddace...	1.665	11,066	33	0.411	4
6abd8b0f...	5.381	3,200	249	0.154	1
55249ca2...	2.241	1,883	44	0.076	1
448e7cae...	1.886	1,729	25	0.067	1

Sorted by tokens in. Red: version 4 in our 28 October copy, the pasted list of restaurants.

Then

click a Trace ID to see its steps

About 11,000 tokens in under version 4 against 1,700 to 3,200 under version 1: every answer reads the pasted list again.

Cost per run hides which answers are expensive. Sort the traces and they come to the top.

What to log per answer, and where Foundry keeps it

What	Why	Where Foundry keeps it
Request	the question, which case	the dataset row; the conversation id
Steps	tools called, in order	the trace: its Execute Tool steps
Generation	model, agent version, tokens	the trace and its agent version
Answer	the text, right or wrong	the evaluation: verdict and reason per case
Cost and time	rupees and seconds	the Traces tab: Estimated cost, Duration

Evaluation results hold the question and the answer text: fine for test cases, not for real customers.

Totals are easy later; rebuilding one answer from totals is impossible.

Break

10 minutes. After the break: health checks on the pipeline, and the faults nothing caught.

3

Data monitoring and drift

Five checks, each with an owner

Types of drift and how to detect them

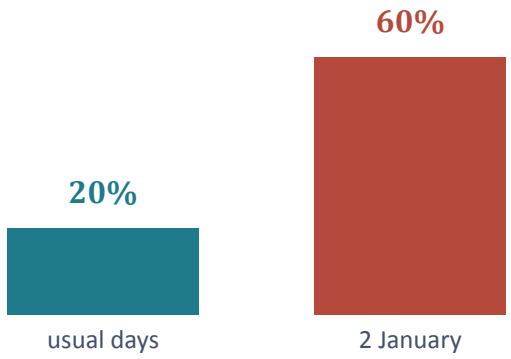
Data	the records change in character	ticket mix, volume, quarantine rate
Prompt	someone edits the prompt	quality check: right answers; labelling cost
Tools and code	a tool returns more, or runs slower	quality check: cost and seconds; traces
Model	the provider updates it behind the same name	the nightly quality check
Source edits	old rows change at the source	nothing today

Your own changes cause most incidents, and they are the easiest to catch.

Drift detection: data, embedding and prompt drift

Data drift

the inputs changed



late_delivery share of the day's tickets

Embedding drift

their meaning changed

distance of the day's average meaning from the last 30 days



no labels needed: it also sees a new kind of ticket

Prompt drift

you changed

prompt v1	prompt v2
$0.000248	$0.001964

one labelling run, 10 texts

7.9 times the cost, 0 of 10 labels moved

record a fingerprint of the prompt with every run, then compare

1. Pick a normal window: the last 30 days

2. Measure one number per day: a share, a distance, a cost

3. Set the limit from normal days, then alert

Data drift: the inputs changed. Embedding drift: their meaning changed. Prompt drift: you did.

Data health checks: limit, owner, first action

Pipeline is up to date	newest raw ticket is handled	data engineer on call
Volume is normal	50% to 200% of the usual day	data engineer on call
Quarantine rate	at most 5% of the day	data owner
Ticket mix	no category moves more than 10 points	support operations lead
Labelling cost	at most 5 times the usual run	ML engineer

Any alert fails the step on purpose: the pipeline run shows red and names the check.

Data quality alert: a bad day of tickets

Copied, not yet processed

ALERT pipeline is up to date
ALERT ticket mix (no labels yet)

Our first version crashed here instead.

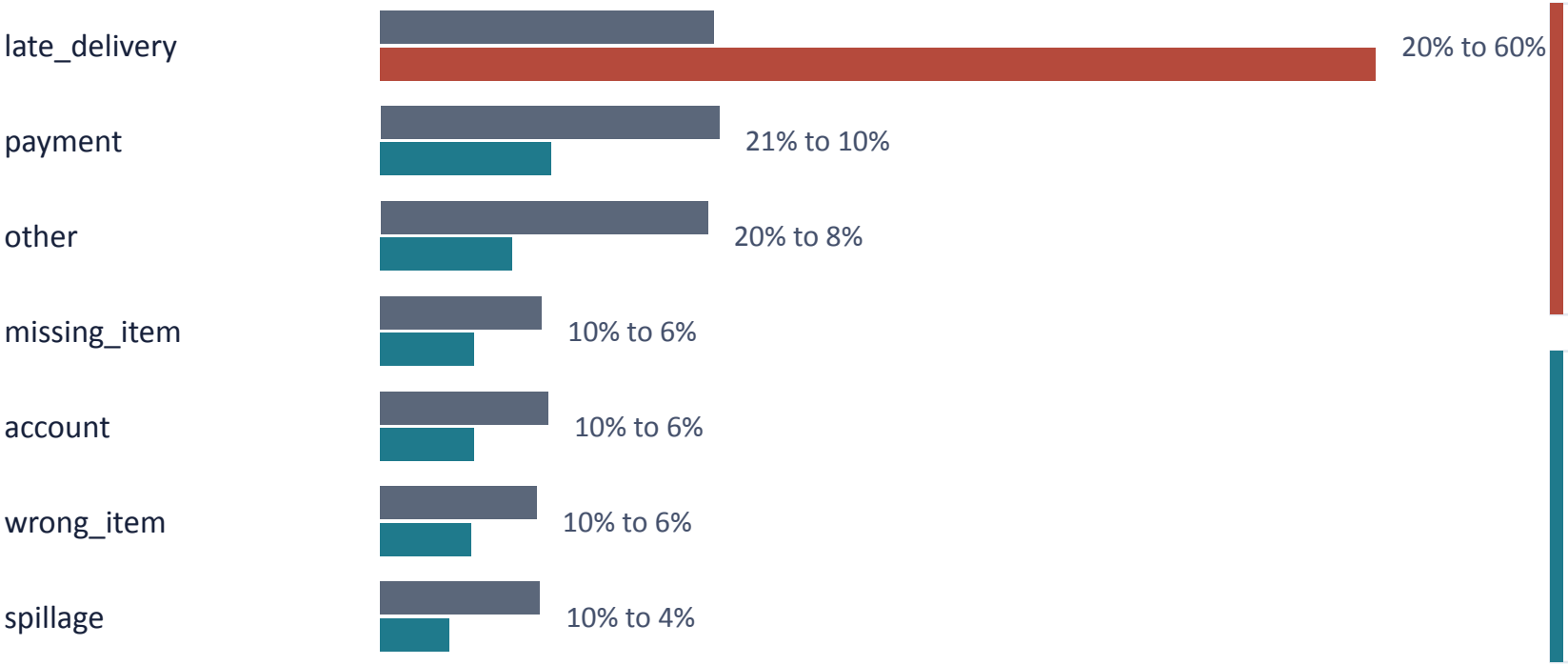
After the pipeline (253 s)

ALERT quarantine rate: 18.3% (110 of 600)
Everything else OK.

Baseline day: quarantine 3.7% (22 of 600), under the 5% limit. The bad day is five times worse.

A health check must never crash when the pipeline is behind: it alerts, then fails the step on purpose.

Distribution drift: a late-delivery surge



Grey: previous 30 days. Colour: 2 January.

ALERT ticket mix

late_delivery +40.2 points

Quarantine 0.5%

The data was clean. Nothing was broken.

Some alerts are not bugs: check what happened that day before blaming the model.

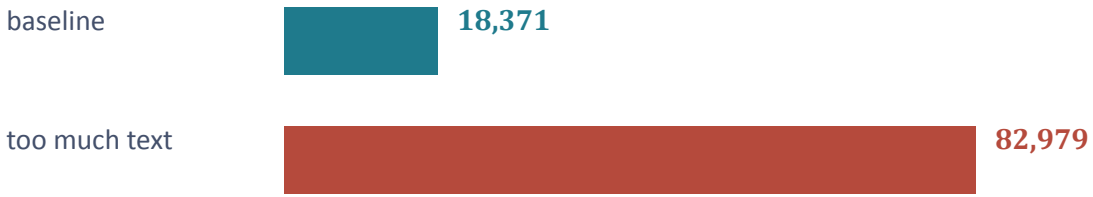
Cost monitoring: from tokens to rupees, and where to watch

One answer, worked out (gpt-4.1-mini: $0.40 per million tokens in, $1.60 out)

1,950 in x $0.40 / 1M	+	102 out x $1.60 / 1M	=	one answer
$0.000780		**$0.000163**		**$0.00094**

one baseline answer from our Traces tab, which showed it as about Rs 0.087: about Rs 92 to the dollar

Tokens for the 7 cases (Lab 10, fault 2)



4.5 times the tokens, no better answers: only the cost signal caught it.

Per answer	Traces tab: Rs 0.06 to 0.16 for our baseline answers
Per test run	the evaluation's tokens: limit 30% above the baseline
Per pipeline step	Lab 9 labels: one call per ticket about $4.97 (estimate); per distinct text $0.000248; re-run $0

Cost is tokens times price: watch it per answer, against a baseline, the same day.

4

Coverage gaps and alerting

Name the gap honestly

Alert design: when, who, first action

```
WHEN  quarantine rate > 5% of the day's tickets
WHO   data owner (support operations)
FIRST read tickets_quarantine for that day: which rule, which channel
```

Alert on symptoms

Rates and shares, not every failed call: a call that is retried is normal.

Give it an owner

Every alert names who acts, and what they do first.

Tune it the same week

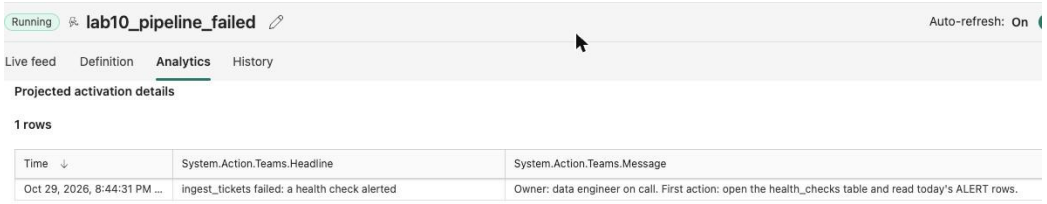
An alert that fires falsely twice gets muted forever.

A signal nobody acts on is worse than none: it trains people to look away.

Demo: a Teams alert with no code, and the MLflow chart

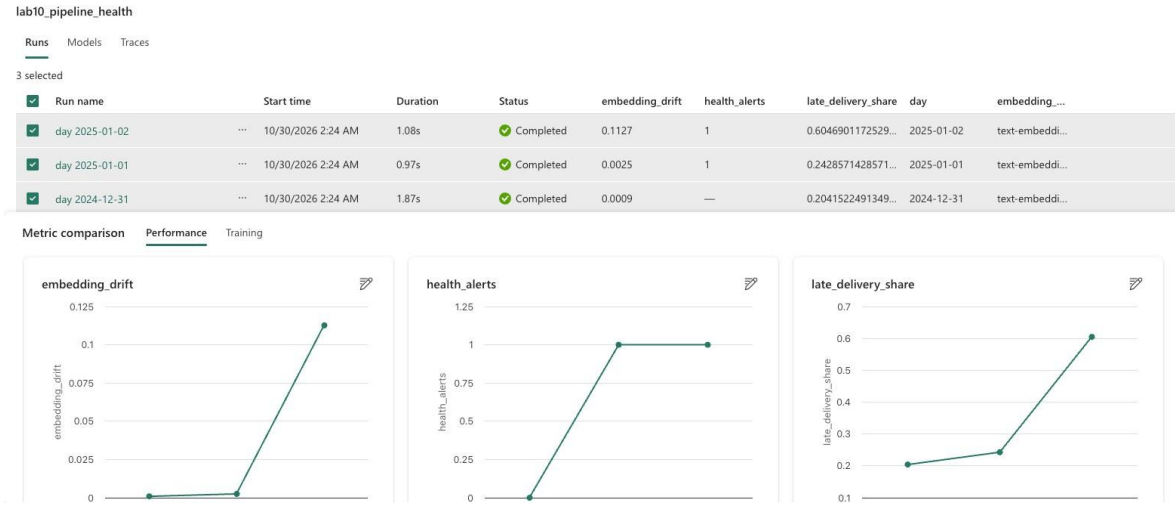
Alert when the pipeline turns red

1. Real-Time > Fabric events
2. Job events > Set alert
3. ItemJobFailed on ingest_tickets
4. Teams message set to send: owner, first action



lab10_alerts: one of the two activations, and the message it was set to send. Fired twice: bad day, late day.

MLflow: three days side by side



The late-delivery day stands out in embedding_drift (0.1127) and late_delivery_share (0.60). The bad day does not: broken data, same meaning.

An alert only helps if it reaches a person. A chart only helps if every day is recorded.

Coverage gaps: two faults no check caught

The December bug

Refund rate 4.25% instead of 4.57%. The 7 cases only ask about March.

The edited old ticket

TKT-0000042: 999.00 at the source, 103.17 in the lakehouse. The watermark never looks back.

Add a case

One business number for a month after October.

Add a compare

Weekly: count and checksum source against lake.

Write the gap down

What your checks do not see is part of the design.

QUICK CHECK

Three questions before the lab

1. The mix alert fires on a day with 0.5% quarantine. Bug, or news?

2. Your nightly quality check costs 3 cents. When would you NOT run it every night?

3. Name one fault in your own work that none of today's checks would catch.

Take answers in chat, then into the lab.

Lab 10

Knowing when it breaks

75 minutes · Foundry portal + your Fabric workspace · lab10_monitoring folder · handbook Lab10_knowing_when_it_breaks.pdf

Build

- A. Agent quality check: evaluation, two baselines, two faults
- B. Traces tab: where one answer's time and tokens went
- C. lab10_0_refresh first, lab10_3_health last; bad day, then a surge
- C. A Teams alert when the pipeline turns red, no code
- D. lab10_4_drift: embedding drift and MLflow, one run per day
- Hand in: what nothing caught, one check each, one alert rule

Hand in

- Which signal caught which fault
- The fault nothing caught, and the check you would add
- One alert rule: when, who, first action

Lab 10 results: which signal caught which

never escalate	**right answers, only just**	4 of 7, limit 5
too much text	**tokens**	82,979, 4.5 times
window (optional code)	**right answers**	13 of 18
slow (optional code)	**time**	26.1 s
bad day	**quarantine rate**	18.3%
late surge	**ticket mix**	+40.2 points
late surge, by meaning	**embedding drift**	0.1127, limit 0.02
coverage (optional code), old edit	**nothing**	write the gap down

ASK THE ROOM

Read two alert rules aloud. Does each name an owner and a first action?

TAKE AWAY

It runs, and it tells you when it breaks

Day 9

Copy what is new, check, label with a cache, run in order, log every run.

Day 10

A quality check on the agent, traces per answer, health checks with owners.

The gaps

Edits at the source and cases outside the test set: named, not hidden.

Next: Day 11

Who is allowed to see what: privacy, entitlements, and the capstone.

Week 6 decides whether this can face a real customer.